

ESM Credit LOS — Frontend Map

Root: frontend/ · **Stack:** React 19 + TypeScript + Vite + React Router v6, Tailwind + Financial Core design tokens (--cr- *). Credit pages are **eagerly imported** (no React.lazy), guarded by ProtectedRoute permission strings, and nested under a shared CreditLayout.

> **i18n & Sentry:** the credit module uses **hard-coded English** (no i18next — global i18n exists in src/i18n/ but only pages/Dashboard.tsx uses it). Sentry (@sentry/react) is wired **only at app level** via a global ErrorBoundary; no credit page/component makes direct Sentry.* calls.

1. Routing (frontend/App.tsx, lines 307–329)

All credit routes nest under one layout route wrapped in ProtectedRoute
requirePermission="credit:read":

```
<Route path="/credit" element={<ProtectedRoute requirePermission="credit:read"><CreditLayout /></ProtectedRoute>}>
  <Route index element={<CreditDashboard />} />
  <Route path="borrowers" element={<BorrowerProfileList />} />
  <Route path="borrowers/new" element={<ProtectedRoute
requirePermission="credit:create"><CreateBorrowerPage /></ProtectedRoute>} />
  <Route path="borrowers/:id" element={<BorrowerProfileDetail />} />
  <Route path="applications" element={<CreditApplicationList />} />
  <Route path="applications/new" element={<ProtectedRoute
requirePermission="credit:create"><CreditApplicationCreate /></ProtectedRoute>} />
  <Route path="applications/:id" element={<CreditApplicationDetail />} />
  <Route path="approvals" element={<ProtectedRoute
requirePermission="credit:approve"><MyApprovals /></ProtectedRoute>} />
  <Route path="financials" element={<FinancialSpreading />} />
  <Route path="analysis" element={<FinancialAnalysis />} />
  <Route path="scorecards" element={<ProtectedRoute
requirePermission="credit:admin"><ScorecardManagement /></ProtectedRoute>} />
  <Route path="rating-bands" element={<ProtectedRoute
requirePermission="credit:admin"><RatingBandAdmin /></ProtectedRoute>} />
  <Route path="committee" element={<CommitteeMeetings />} />
  <Route path="committee/:meetingId" element={<CommitteeMeetingDetail />} />
  <Route path="m/committee/:meetingId" element={<ProtectedRoute
requirePermission="credit:approve"><CommitteeMobileVote /></ProtectedRoute>} />
  <Route path="m/approvals" element={<ProtectedRoute
requirePermission="credit:approve"><MobileApprovalInbox /></ProtectedRoute>} />
  <Route path="m/applications/:id" element={<CreditApplicationMobileSummary />} />
  <Route path="collateral" element={<CollateralManagement />} />
  <Route path="reports" element={<CreditReports />} />
  <Route path="group-exposure" element={<GroupExposurePage />} />
</Route>
```

Guards: every route inherits credit:read from the layout; sub-routes add credit:create (borrowers/new, applications/new), credit:approve (approvals, m/committee, m/approvals), credit:admin (scorecards, rating-bands). **No lazy loading.** Route-level feature flags are not used — flags are checked inside CreditApplicationDetail via useCreditFeatureFlags.

2. Pages

frontend/pages/credit/

File	Purpose / key features
<code>`CreditDashboard.tsx`</code>	Role/lane-aware dashboard. Uses <code>`dashboardApi`</code> + <code>`branchApi`</code> + <code>`ApprovalInbox`</code> ; renders <code>`AttentionStrip`</code> , <code>`LaneSwitcher`</code> , and one of <code>`RmLane`</code> / <code>`ApproverLane`</code> / <code>`ManagerLane`</code> (via <code>`useCreditLane`</code>)
<code>`CreditApplicationCreate.tsx`</code>	Multi-step creation wizard (<code>`/credit/applications/new`</code>); uses <code>`financialApi`</code> , <code>`retailIncomeApi`</code> , <code>`branchApi`</code> ; drafts via <code>`saveApplicationDraft`</code>
<code>`CommitteeMeetingDetail.tsx`</code>	Committee workspace — agenda, members, attendance, votes, quorum, decision finalization (<code>`committeeApi`</code>)
<code>`CommitteeMobileVote.tsx`</code>	Mobile ($\leq 768\text{px}$) voting; sticky header, comment min-length 10
<code>`MobileApprovalInbox.tsx`</code>	Mobile approval inbox; uses <code>`dashboardApi`</code> + <code>`ApprovalInbox`</code> ; approve/reject via <code>`validateApprovalDecision`</code> / <code>`buildApprovalPayload`</code>
<code>`CreditApplicationMobileSummary.tsx`</code>	Mobile card summary
<code>`CreditReports.tsx`</code>	Pipeline / exposure / turnaround reports (<code>`reportsApi`</code>)
<code>`GroupExposurePage.tsx`</code>	Related-party group exposure (<code>`relatedPartyGroupApi`</code>)
<code>`PersonalFastView.tsx`</code>	**PERSONAL_FAST** processing-lane specialized application view; reuses tab components + <code>`EditBorrowerModal`</code>
<code>`RejectionBanner.tsx`</code>	Inline rejection-reasons banner (<code>`rejectionApi`</code> ; reason-code label map)

frontend/pages/ **(credit module, outside credit/)**

CreditApplicationList.tsx, CreditApplicationDetail.tsx (see §3), BorrowerProfileList.tsx, BorrowerProfileDetail.tsx (borrower360 workspace), CreateBorrowerPage.tsx (multi-step NewBorrowerWizard), CommitteeMeetings.tsx, MyApprovals.tsx (ApprovalInbox), CollateralManagement.tsx, FinancialSpreading.tsx, FinancialAnalysis.tsx, ScorecardManagement.tsx, RatingBandAdmin.tsx.

3. Application Detail (frontend/pages/CreditApplicationDetail.tsx, 1,152 lines) + Tabs

The detail page is an **"Application 360 Workspace"**. It reads `?tab=/?area=` query params (legacy tab params auto-mapped to 360 equivalents via `TAB_TO_TAB360`), uses `useApplicationLane(id)` + `useCreditFeatureFlags()` to build the tab set, and renders the header (`ApplicationWorkspaceHeader`), journey stepper, workspace navigation, and per-area workspaces. Tab switching goes through `useDirtyFormGuard` (dirty-form confirm). `PersonalFastView` is rendered when the app's lane is `PERSONAL_FAST`.

Tab enablement is three-layered (frontend/pages/credit/creditUtils.ts: `getVisibleTabGroups`):

- 1 Backend returns lane/tabs via `creditService.getApplicationTabs(id)` (GET `/credit/applications/:id/tabs`) → `src/hooks/useApplicationLane.ts` (lane, reason, `requiredApproverCount`, tabs).
- 2 Frontend re-filters locally via `getVisibleTabGroups(advancedMemo, borrowerType, applicationState, featureFlags, lane)`.

- 3 Filter rules: advancedOnly groups require the credit:advanced_memo flag; per-tab/group feature flags (GROUP_FEATURE_FLAGS, TAB_FEATURE_FLAGS); **state** gating (g.states); **lane-based** filtering (isTabVisibleForLane); retail relabels parties → "Guarantors & Parties".

ApplicationHorizontalTabs.tsx renders the scrollable grouped tab bar (S1–S7 + ADV + meta groups) with phase-completion dots and a Documents count badge. getPhaseCompletion models 7 completion sections (S1 Loan Request → S7 Decision, META optional).

frontend/pages/credit/tabs/ (360 merged tabs)

Tab	Notes
`ApplicationDetailsTab`	Core application fields / details editing
`ApprovalsTab360`	Wraps `sections/ApprovalsTab`
`BorrowerProfileTab`	Borrower profile + `CaMemoSection` + FATCA/CRS
`CaMemoPreviewTab`	CA Memo preview/export (`downloadCaMemo`, `pollPdfJob`)
`CollateralGuaranteesTab`	Wraps `sections/CollateralTab`
`ConditionsOfferTab`	Wraps `sections/ConditionsTab` (conditions & waivers)
`CreditBureauComplianceTab`	Bureau/compliance wrapper → `CreditChecksTab`
`CreditChecksTab`	CCRIS/CTOS checklist + AML (`bureauChecklistApi`)
`CustomerProfileTab`	Retail borrower profile + FATCA/CRS (`RetailBorrowerProfile`)
`DisbursementTab`	Disbursement orders/readiness (`disbursementApi`)
`DocumentsTab`	Document list/upload/verify + requirement checklist
`FinancialProfileTab`	Retail income profile (`retailIncomeApi`)
`FinancialsTab`	Financial statements/ratios (`financialApi`)
`LoanRequestTab`	Loan request / facilities / request-items editing
`RiskAssessmentTab`	Wraps `sections/RiskScoreTab`
`SignoffTab`	Sign-off roles (PREPARED_BY/REVIEWED_BY/CONCURRED_BY) via `signoffApi`
`TimelineAuditTab`	Wraps `sections/AuditTab` + `ApplicationComments`

frontend/pages/credit/tabs/sections/

Granular sub-panels composed into the 360 tabs: AccountConductTab, ApprovalsTab, AuditTab, CollateralTab, ConditionsTab, CorporateBorrowerProfile, CounterpartiesTab, ForwardLookingRiskTab, GuarantorFinancialAssessmentTab, IndustryOutlookTab, LooSection, PartiesTab, PaymentCapabilityTab, PricingWorksheetPanel, ProfitabilityWalletTab, QualitativeAssessmentTab, RequestsFacilitiesTab, RetailBorrowerProfile, RetailFacilitiesTab, RetailIncomeTab, RiskMitigatorsTab, RiskRatingEclTab, RiskScoreTab, SecurityGuaranteesTab, SmeBorrowerProfile, SmeFinancialsTab, SummaryTab.

4. Components (frontend/src/components/credit/)

borrower360 workspace (borrower360/): BorrowerWorkspaceHeader, Borrower360Header, BorrowerRelationshipSnapshot, BorrowerReadinessStrip, BorrowerNextActions, IncomeEditModal,

BorrowerOverview, AssessmentReadinessChecklist, BorrowerKpiBand, ExposureFacilitiesTab, BureauUploadModal, BorrowerProfileTab, BorrowerExposureSnapshot, BorrowerApplicationSummary, BorrowerActivityTimeline, DocumentChecklistSummary, CorporateOverview, RetailOverview, RiskAssessmentResultCard, primitives.

Top-level shared components: ApplicationComments, ApprovalMatrixApplicabilityPanel, ApprovalChainPanel, ApprovalPackPreview, ApprovalQuickView, ApplicationTimeline, AutosaveTextField, BulkDocumentUpload, BusinessProfileSection, BorrowerSummaryCard, CalculationBreakdownPanel, CaMemoSection, CollapsibleSection, CommitteeWidget, CreditTable, DocumentUpload, EditBorrowerModal, EvidenceMappingPanel, FatcaCrsSection, FinancialCharts, FinancialProfileSummaryStrip, FinancialRiskIndicatorsPanel, IntegrationModeBanner, JointBorrowerSection, NewBorrowerWizard, PartyFormModal, ProgressOverlay, RatiosAndTrendsSection, ReadinessChecklistModal, RecommendationSection, RiskBadge, S7ProcessBanner, ScoreOutdatedBanner, SectionStates, SlaBreachWidget, StateBadge, UserAssignChip, ValidationIndicators, ValidationOverridePanel.

Sub-namespaces:

- dashboard/ — lane components: RmLane, ApproverLane, ManagerLane, LaneSwitcher, DecisionCard, DecisionActions, AttentionStrip, PriorityWorkQueue, NextActionsPanel, OperationalAlerts, useCreditLane.
- detail/ — Application 360 workspace: ApplicationWorkspaceHeader/Navigation, ApplicationPartiesWorkspace, FinancialsWorkspace, RiskComplianceWorkspace, AssessmentRecommendationWorkspace, DecisionCompletionWorkspace, ApplicationJourneyStepper, ApplicationHorizontalTabs, ApplicationOverviewTab, ApplicationStatusWidget, ApplicationSlaWidget, ApplicationTeamWidget, ApplicationSummaryPanel, ApplicationReadinessPanel, ApplicationPendingTasks, ApplicationHealthPanel, ApplicationNotesWidget, ApplicationCustomerInsights, ApplicationCollaborationPanel, ApplicationAlertsPanel, SectionCompletionHeader, CreditDecisionSummaryCard, ApplicationKpiRow, ApplicationNextAction, ApplicationSectionIndex.
- applications/ (list UI), approvals/, borrowers/, create-borrower/, new-application/.

5. Services & Types ([frontend/src/services/](#))

`credit.service.ts` (3,776 lines) — the module's single API surface

Exports a **default** `creditService` object with **223 async methods** calling `ApiClient`. Also exports **37 per-domain API objects** (`financialApi`, `trendApi`, `branchApi`, `exposureApi`, `scorecardApi`, `scoringApi`, `committeeApi`, `collateralApi`, `guaranteeApi`, `conditionApi`, `dashboardApi`, `reportsApi`, `profitabilityApi`, `walletShareApi`, `keyCounterpartyApi`, `bureauCheckApi`, `industryAssessmentApi`, `riskAssessmentApi`, `rmdIssueApi`, `esgApi`, `sicrApi`, `signoffApi`, `utilisationApi`, `piiRevealApi`, `bureauChecklistApi`, `fatcaCrsApi`, `retailIncomeApi`, `qualitativeAssessmentApi`, `disbursementApi`, `looApi`, `rejectionApi`, `policyLimitApi`, `amlRescreenApi`, `relatedPartyGroupApi`, `commentApi`, `scoreStatusApi`, `creditRecommendationApi`), plus `normalizeCreditDocument()`.

`creditService` **method groups (representative):**

- **Borrowers:** listBorrowerProfiles, getBorrowerProfile, getBorrower360Summary/Activity, updateBorrowerIncome/CreditProfile, createBorrowerProfile, checkDuplicateBorrower, searchBorrowers, runKyc, runAml, markBorrowerKycVerified, calculateBorrowerRiskScore, getBorrowerRiskHistory, getBorrowerStats, getBorrowerOnboarding, createDirector/Shareholder/Ubo, checkBorrowerIdentity, requestDuplicateException, decideDuplicateException.
- **Documents:** listDocuments, listDocumentRequirements, getChecklistSummary, uploadDocument, getDocumentDownloadUrl, getEvidenceMapping/saveEvidenceMapping, verifyDocument/rejectDocument/deleteDocument.
- **Applications:** listApplications, getApplication, createApplication, draft CRUD, updateApplication, cloneApplication, checkReadiness, checkEsignReadiness, listFeatureFlags, getPublicFeatureFlags, previewCaMemoHtml, getApplicationLane, reEvaluateLane, getApplicationTabs, transitionApplication, getApplicationTransitions, getApplicationAudit, downloadCaMemo, getApprovalPackHtml, downloadApprovalPackPdf.
- **Facilities / CA Memo:** facility CRUD, getPricingWorksheet/upsertPricingWorksheet/computePricingPreview, listRequestItems CRUD, getExposureSummary/upsertExposureSummary.
- **Credit / ECL / Projections:** listExternalRatings, listEclSnapshots/Forecasts, getCashflowProjection, listSensitivityScenarios, getActiveRatingBands, getExposure, getPresentation.
- **Parties & financials:** party CRUD, financial-statement CRUD (validateBalanceSheet, computeRatios, listRatios, getTrends).
- **Approvals & decision:** listApprovals, submitApproval, getApprovalMatrixApplicability, listApprovalMatrices, lookupApprovalAuthority, listRejectionReasonCodes, getDashboard, listScoreRuns, executeScore, requestScoreOverride, overrideScore.
- **Committee / Collateral / Conditions:** listMeetings/getMeeting/castVote/finalizeDecision/generateMemo, addValuation/addLien/addInsurance, completeCondition/waiveCondition, checkCpCompletion.

credit.types.ts (59 lines) — deliberately small DTO types

ApprovalInboxItem, ApprovalInboxExclusion, ApprovalInbox, CreditApiResponse<T> (the standard {status, data} envelope). Documents why fields differ from CreditApplication (prevents the My-Approvals crash); currentState is intentionally string to avoid a circular import with ApplicationState.

frontend/src/types/credit-ui.types.ts (148 lines)

Borrower list/query/response types, BorrowerSegment, BorrowerLifecycleStatus, BorrowerDataQuality, DuplicateIdentityResult, DuplicateException*, DashboardAttention/NextAction/WorkItem, CreditOfficerDashboard.

Types/enums defined inline in credit.service.ts

ApplicationState (DRAFT→WITHDRAWN/REFERRED_BACK), CreditProductType, FacilityType (incl. Islamic RWC_I, LC_I, BG_I, ICMT_I, TRUST_RECEIPT...), CaRequestType, CurrencyCode, ApprovalDecision, ApplicationType, AccountClassification, AccountStrategy, RiskRating, FinancialStatementType/Period/Status, RatioCategory, BorrowerProfileStatus, DocumentType/Status, AmlRescreenOutcome/Action, MeetingStatus/Type/MemberRole/VoteChoice/DecisionType, CollateralType/LienStatus/SecurityCategory, GuaranteeType, ConditionStatus/Category,

CounterpartyRole, BureauProvider, RiskCategory, Esg*, SocrTriggerType, SignoffRole, FatcaEntityClassification, plus interfaces BorrowerProfile, Borrower360Summary, CreditApplication, CreditFacility, CreditApproval, CreditScoreRun, CommitteeMeeting, Collateral, Guarantee, ConditionPrecedent, FinancialStatement, DisbursementOrder, RelatedPartyGroup, GroupExposureData, ApplicationComment, PolicyLimit, CreditRecommendation, etc.

creditAi.service.ts

AI-assisted credit service (companion to credit.service).

6. Navigation & RBAC

- **Global nav:** src/components/layout/navConfig.ts:33 — { to: '/credit', label: 'Credit', icon: 'account_balance', group: 'tools', show: hasAnyPermission(user, ['credit:read']) }. Line 19 — /approvals shown if credit:approve OR request:approve.
- **Module sub-nav:** src/components/CreditNav.tsx renders the horizontal nav (Dashboard, Borrowers, Applications, Group Exposure, My Approvals, Scorecards, Analysis, Spreading, Collateral, Reports) inside src/components/CreditLayout.tsx, each item permission-gated via hasPermission. Overflow collapses into a "More" dropdown. **Committee / rating-bands / financials are reached only by direct route, not the top nav.**
- **Permission helpers:** src/utils/permissions.ts (hasPermission/hasAnyPermission); per-page writes also gated by credit:write (e.g. canWrite in the detail page).
- **Feature flags:** src/hooks/useCreditFeatureFlags.ts → creditService.getPublicFeatureFlags() (/credit/feature-flags/public), maps {key:boolean} like credit:advanced_memo, credit:ecl, FATCA/CRS; consumed by getVisibleTabGroups. App-level flags via src/lib/featureFlags (isFeatureEnabled('kb')).

7. Data-flow pattern

Page/Component → creditService (or domain api) → apiClient → /api/v1/credit/... → backend controller → service → Prisma → JSON → component state.

Worked example: the **application 360 workspace** loads via getApplication (GET /applications/:id); each area fetches its own slice (facilities, risk-assessment, score, etc.) and re-renders on the shared application object. The lane endpoint (GET /applications/:id/tabs) + frontend getVisibleTabGroups together decide which tabs render.